ENGINEERING HIRING PACK

iOS Intern Assignment

TasteIndia — Indian cuisine discovery

> **Your track** Build one native iOS application using Swift + SwiftUI. This document is standalone; candidates are not expected to read or implement the other platform brief.

Scope	Expectation
Theme	Browse Indian recipes, search and filter the list, inspect complete recipe details, and save favourites.
Timebox	8–12 focused hours over 3–5 calendar days.
Primary evidence	Native list UI, state ownership, public REST integration, filtering, details navigation, persistence, and resilient failure handling.

Prepared September 2026

1. Candidate brief

Build TasteIndia, a small Indian-cuisine discovery app powered only by TheMealDB's publicly documented API. The core experience is a polished recipe List View with filters and a Details View that presents every meaningful, non-empty recipe field returned by the API.

API rule Use TheMealDB V1 with the educational test key 1. Do not add private, scraped, reverse-engineered, paid, or partner-only APIs. The submitted app must remain reviewable using checked-in fixtures if the public service is temporarily unavailable.

AI usage AI tools are allowed for research, explanation, debugging, test ideas, and assisted implementation. A submission that is 100% agent-generated, unreviewed, or not understood by the candidate will not be accepted. The candidate must own every submitted decision and be able to explain, test, and modify the code.

What to build

1. Fetch the Indian meal collection from TheMealDB and display it in a native, scrollable List View.
2. Provide search, filters, sorting, applied-filter feedback, and a clear-all action.
3. Open a recipe by stable meal ID and display complete user-facing details from the lookup response.
4. Preserve list position, search text, filters, and sort order when navigating to details and back.
5. Allow a meal to be favourited from the list or details screen and persist favourites across relaunch.
6. Handle loading, empty, partial-data, image-failure, offline, timeout, and retry states deliberately.

Required List View and filters

- Each row must use strMeal as the title, strMealThumb as the image, idMeal as stable identity, and an accessible favourite control. Show category or other enrichment only after it has been legitimately loaded.
- Required filters: meal-name search, category, main ingredient, and favourites-only. Required sort: name A–Z and Z–A.
- TheMealDB V1 does not provide one endpoint that combines area, category, and ingredient. Intersect meal IDs locally or use a documented cached-detail strategy; keep Indian-area membership as a hard boundary.
- Show active filters, result count, empty-filter results, and Clear all. Debounce or locally apply search so rapid input does not create stale UI.
- Avoid uncontrolled N+1 work: bound concurrent detail lookups, cache responses, and never start network requests from an individual row's render/body function.

Required navigation flow

Recipes ⟶ Recipe details. A separate Favourites destination must display saved meals and open the same Details View. Back/Up must restore the exact list context rather than resetting search, filters, sort, or scroll position.

Screen	Required behavior
Recipes	Native list with thumbnails, names, favourites, loading/empty/error/retry, result count, and preserved scroll state.

Screen	Required behavior
Filters	Search text, category, main ingredient, favourites-only, sort order, active state, Apply/Clear behavior, and accessible controls.
Recipe details	Hero image, name, category, area, ingredient/measure pairs, instructions, tags, source link, and video link when non-empty.
Favourites	Persist saved IDs across relaunch; support removal and the same ID-based details route.

2. TheMealDB API contract

Official documentation: https://www.themealdb.com/api.php. The test key 1 is permitted for development and educational use. This hiring exercise is not an App Store or Play Store release; candidates must still read the provider's current terms and attribute the source.

Purpose	Public endpoint	Use in the app
Indian meals	https://www.themealdb.com/api/json/v1/1/filter.php?a=Indian	Authoritative base set for the List View and Indian-area boundary.
Full details	https://www.themealdb.com/api/json/v1/1/lookup.php?i={mealId}	Load the selected meal's complete detail record by stable ID.
Categories / areas	https://www.themealdb.com/api/json/v1/1/list.php?c=list https://www.themealdb.com/api/json/v1/1/list.php?a=list	Populate documented filter choices or validate returned values.
Ingredient filter	https://www.themealdb.com/api/json/v1/1/filter.php?i={ingredient}	Intersect returned IDs with the Indian base set.
Category filter	https://www.themealdb.com/api/json/v1/1/filter.php?c={category}	Intersect returned IDs with the Indian base set.
Name search	https://www.themealdb.com/api/json/v1/1/search.php?s={query}	Optional remote search; keep only Indian results, or search the loaded Indian set locally.

> **Public API only** No API registration, secret, or company-provided credential is required for this educational assignment. Do not commit a supporter key. A fixture may mirror the public response for tests and offline review, but the running app must demonstrate at least one real TheMealDB GET request.

Details View: required field mapping

TheMealDB field	Presentation rule
idMeal	Stable identity and route argument; do not display as primary content.
strMeal / strMealThumb	Title and responsive hero image with loading and failure treatment.
strCategory / strArea	Display as metadata when non-empty; area should remain Indian for the required collection.
strIngredient1…20 + strMeasure1…20	Normalize into ordered ingredient rows; trim whitespace and omit empty pairs.
strInstructions	Readable multi-paragraph instructions; preserve meaningful line breaks.
strTags	Split comma-separated tags into accessible chips or labels; omit when null/blank.

TheMealDB field	Presentation rule
strSource / strYoutube	Open only valid URLs using platform conventions; label links descriptively.
Other fields	Do not render raw JSON or empty labels. Present every additional meaningful user-facing field returned by the chosen V1 response and document intentional omissions.

3. iOS technical expectations

- Swift and SwiftUI with a stated deployment target (recommended: iOS 17+).
- NavigationStack with typed or centrally defined destinations that pass meal IDs rather than large models.
- URLSession, async/await, Codable DTOs, explicit HTTP status validation, timeouts, cancellation, and safe URL construction.
- @Observable or an appropriate ObservableObject-based state holder with clear ownership; @State only for local ephemeral UI state.
- SwiftData, Core Data, or a proportional file store for favourites and optional response cache.
- Dependency injection that replaces meal service and persistence with deterministic test doubles.
- At least four focused tests: decoding/ingredient normalization, filter intersection, latest-query/list state, and favourites or navigation restoration.

4. iOS do’s

- Model initial, loading, loaded, empty, stale, and failed states explicitly.
- Use List or LazyVStack deliberately, stable Identifiable meal IDs, and reusable row components.
- Cancel stale remote searches and prevent older responses from replacing newer filter state.
- Keep networking, filtering, image loading, and persistence side effects out of a SwiftUI view body.
- Preserve List position, search text, filters, and sort order across NavigationStack transitions.
- Normalize the 20 ingredient/measure fields into a clean domain collection before rendering.
- Cache or deduplicate detail requests and place a deliberate bound on concurrent enrichment calls.
- Treat API DTOs, domain meals, filter state, and persisted favourites as separate concerns.
- Use Dynamic Type, semantic labels, sufficient contrast, accessible filter controls, and descriptive links.
- Use redacted fixtures and previews for loading, empty, error, long instructions, missing images, and details states.
- Open source/video URLs with platform-safe APIs and validate the URL first.
- Explain every third-party package and prefer platform APIs when they are sufficient.

5. iOS don’ts

- Do not force-unwrap decoded fields, route IDs, image URLs, or external links.
- Do not start requests or mutate shared state directly from body.
- Do not recreate an observable model whenever a view redraws.
- Do not use list indexes as identity in List or ForEach.

- Do not trigger one detail request per visible row without caching, deduplication, and concurrency control.
- Do not pass a full Meal object through route strings; pass idMeal and load from a repository/cache.
- Do not reset applied filters or scroll position after returning from details.
- Do not show blank ingredient rows, empty labels, raw JSON keys, or malformed URLs.
- Do not claim combined server-side filtering when results were intersected locally.
- Do not hide all features inside one oversized view or one all-purpose view model.
- Do not make favourites depend on network availability.
- Do not add scraped, private, paid, or partner-only data sources.

6. Reliability and acceptance criteria

Scenario	Expected behavior
Initial load	Show a deliberate loading state, then the Indian recipe list or a recoverable error.
No results	Explain that no recipes match; preserve filters and offer Clear all.
Provider error	Show a human-readable error and retry; never remain on an indefinite spinner.
Rapid search	Only the latest search/filter state may update visible results.
Combined filters	Intersect stable IDs correctly, retain the Indian boundary, and show active filters/result count.
Partial details	Omit blank fields safely while preserving every meaningful non-empty field.
Image failure	Keep row/details usable with an accessible placeholder and no layout collapse.
Relaunch	Persisted favourites remain available; tests do not require the public API to be online.

Accessibility and responsible use

- Support large text/font scaling, light and dark appearance, and platform screen-reader semantics.
- Do not rely on colour alone for selected filters, favourites, errors, or loading state.
- Give meal images meaningful context without redundantly announcing the visible title.
- Label source and video actions descriptively; do not expose a raw URL as the only accessible name.
- Attribute TheMealDB, respect its current educational-use terms, and avoid unnecessary request volume.

7. Submission package

- A complete Git project must be shared at the end of each working day with its .git history intact. Start Git before implementation and commit meaningful increments throughout the work.
- Use focused, descriptive commits that communicate intent. A single end-of-day dump, generated-only history, or unexplained bulk commit is not considered proper evidence of the candidate's process.
- Publishing on GitHub is not required. The repository may be transferred as a private archive, Git bundle, company-hosted repository, or another agreed private method, provided reviewers receive the full commit history.
- Do not commit API secrets, personal data, build products, dependency caches, or IDE-specific noise; include an appropriate ignore file.

- README with iOS toolchain versions, setup, architecture, route map, endpoint choices, filtering/intersection strategy, cache policy, assumptions, tradeoffs, known issues, time spent, and AI/tool-use disclosure.
- A 2–4 minute recording showing list loading, search, combined filters, details, favourites, error/retry, and state restoration after navigation or relaunch.
- Tests that run offline using representative TheMealDB fixtures and documented commands.
- Attribution for TheMealDB, images, fixtures, and third-party libraries.

AI usage and candidate ownership

- Disclose the AI tools used and briefly identify where they materially influenced code, tests, design, documentation, or debugging.
- Review, adapt, and test AI-assisted output before committing it; the candidate remains responsible for correctness, security, accessibility, and maintainability.
- Be prepared to explain any file or architectural decision and make a small change during review without delegating the exercise to an agent.
- Fully autonomous agent execution that replaces the candidate's engineering work, produces code the candidate cannot explain, or results in an unreviewed 100% agentic submission is not accepted.

Candidate checklist

- Fresh clone builds using the README.
- The app makes at least one real GET request to TheMealDB's public V1 API.
- The Indian base set remains authoritative when category or ingredient filters are combined.
- List search, filters, sort, result count, Clear all, and state restoration work.
- Details present every meaningful non-empty field and omit empty ingredient/measure rows.
- Favourites work offline and survive relaunch.
- Tests pass without the public API being online.
- The shared Git project includes the full, incremental commit history and does not need to be published on GitHub.
- AI assistance is disclosed; the candidate understands, can explain, and can modify all submitted code.

8. Evaluation rubric — 100 points

Category	Pts	Strong evidence
iOS UI craft	20	Idiomatic list/details UI, hierarchy, accessibility, image states, reusable components.
State & architecture	15	Clear ownership, immutable/unidirectional state, repositories, persistence, proportional design.
TheMealDB integration	20	Accurate DTO mapping, ingredient normalization, HTTP errors, cancellation, caching, fixtures, attribution.
Search & filtering	15	Correct Indian-boundary intersections, active state, sorting, empty results, clear-all, latest-query safety.
Navigation	15	ID-based routes, correct Back/Up, preserved list context, shared details from Recipes and Favourites.
Reliability & tests	10	Edge states and focused deterministic tests that protect meaningful risks.
Communication	5	Accurate setup, Git history, assumptions, tradeoffs, API limitations, and honest AI/tool-use disclosure.

Reviewer prompts

1. Show how the app keeps category or ingredient results inside the Indian base set.
2. Where is the source of truth for search, filters, sort order, and favourites?
3. How are repeated detail requests deduplicated and concurrent enrichment calls bounded?
4. How are ingredient1…20 and measure1…20 normalized and tested?
5. What state is restored when returning from details or recreating the screen?
6. Pick one commit and explain why that change was separated at that point in the implementation.
7. Identify one AI-assisted contribution, explain how it was reviewed, and modify it during the discussion.
8. Pick one test and explain the production risk it protects against.